Real-time Collaboration Whiteboard

Field: Web Development (Full-stack, real-time) **Difficulty**: Medium **Duration**: 8 weeks **Team size**: 1–2 interns

Context

Tools like Excalidraw, Miro, and FigJam are everywhere. They look simple but hide non-trivial engineering: real-time sync, conflict resolution, cursor presence, offline edits. The intern will build a small but real collaborative whiteboard, learning WebSockets, state synchronization (CRDT or OT), and modern frontend architecture.

Objectives

- Multi-user whiteboard accessible via a shared URL.
- Real-time sync of shapes, text, and cursors.
- Persistence of boards and authentication.
- Production-grade deployment with a public URL.

Mandatory part

1. Drawing surface

- Infinite canvas, pan and zoom.
- Tools: rectangle, ellipse, freehand, arrow, text, eraser, select/move.
- Undo/redo (local).
- Export as PNG and JSON.

2. Real-time collaboration

- Multiple users on the same board see each other's changes within ~200 ms.
- Live cursors with username and color.
- Conflict-free concurrent edits — pick one approach and justify it:
  - **CRDT** (Yjs, Automerge) — recommended.
  - **OT** (custom or sharedb).
  - Last-write-wins with vector clocks (acceptable for shapes).
- Handle disconnection and reconnection without losing local edits.

3. Persistence & rooms

- Boards saved server-side.
- Each board has a unique URL.
- Public / private boards.

4. Authentication

- Email + password, or OAuth (Google/GitHub).
- "My boards" page listing the user's boards.
- Sharing: invite by email or via shareable link with permission level (view / edit).

5. Deployment

- Deployed on a public URL (Fly.io, Railway, Render, or a personal VPS).
- HTTPS, basic rate limiting, environment-based config.

Bonus

- Voice chat (WebRTC) between users on the same board.
- Embed images and PDFs as background.
- Sticky notes with reactions.
- AI assist: "summarize this board", "convert handwriting to text" via a Claude/OpenAI API call.
- Mobile-friendly version with touch support.
- Versions/history with a "rewind" slider.

Tech stack (suggested)

- **Frontend**: React + Konva / Fabric.js / Pixi, or plain Canvas API.
- **State sync**: Yjs + y-websocket (recommended) or a custom WebSocket layer.
- **Backend**: Node (Express / Fastify) or Go.
- **DB**: Postgres for metadata, Redis for pub/sub, S3-compatible blob for board snapshots.
- **Auth**: Lucia, Auth.js, or roll your own with JWT.

Deliverables

- Public deployed URL.
- Git repo with `README.md` covering architecture and a sequence diagram of a collaborative edit.
- A short demo video (~3 min) showing two browsers editing the same board.
- A "design choices" doc justifying the sync algorithm picked.

Weekly plan

Week	Goal
1	Project setup, canvas with basic shapes, pan/zoom.
2	All drawing tools, undo/redo, local-only.
3	WebSocket layer, broadcast simple events between clients.
4	Integrate CRDT (Yjs) or OT. Live cursors.
5	Auth, board persistence, rooms, sharing.
6	Deployment, HTTPS, env config, basic monitoring.
7	Polish UX, fix sync bugs, add 1–2 bonus features.
8	Stress test with 5+ concurrent users, demo prep.

Evaluation grid

Criterion	Weight
Drawing UX (latency, feel)	15%
Real-time sync correctness (no lost updates, no duplicates)	25%
Auth + persistence + sharing	15%

Deployment + reliability	15%
Code quality, architecture, tests	20%
Demo + design doc	10%

Resources

- Yjs docs: https://docs.yjs.dev/
- *Local-first software* — Ink & Switch.
- Excalidraw source: https://github.com/excalidraw/excalidraw
- Liveblocks blog posts on multiplayer state.

Prerequisites

- Comfortable with JS/TS, modern frontend framework.
- Has heard of WebSockets and event-driven architectures.